Crash Experiment Report

1. Failure Version Results

A. Heavy workload problem

In the failure version, `/products/search/heavy` had no protection, so expensive requests were allowed to execute without restriction. Under concurrent load, this caused severe service degradation.

The heavy endpoints showed:

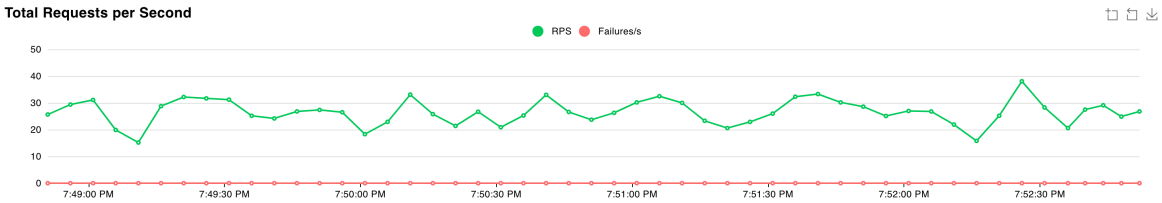
- `median`: about **16–17 s**
- `p95`: about **28–31 s**
- `average`: about **17–18 s**
- `max`: up to **42–54 s**

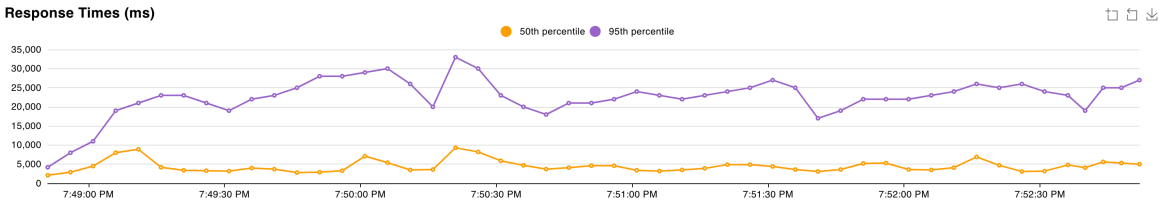
More importantly, the heavy workload also hurt the lightweight traffic. The `/products/search/light` endpoints, which should normally be fast, were dragged up to:

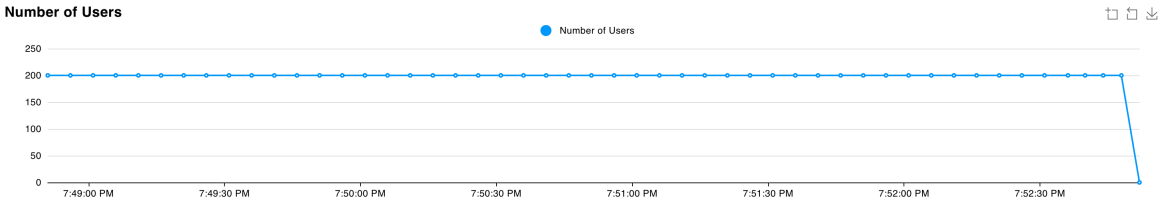
- `median`: about **2.9–3.1 s**
- `p95`: about **8.6–9.2 s**
- `average`: about **3.5–3.8 s**

The aggregated throughput was only about **26.8 RPS**. Even though there were no explicit HTTP failures, this clearly showed an overload condition: heavy requests monopolized service capacity and degraded the responsiveness of normal requests.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search/heavy?q=alpha	273	0	16000	31000	42000	17639.45	892	54413	2161.59	0.9	0
GET	/products/search/heavy?q=beta	276	0	16000	28000	35000	17081.07	4197	42370	2061.57	1.2	0
GET	/products/search/heavy?q=books	301	0	16000	29000	37000	16955.31	3126	48764	2061.55	1.5	0
GET	/products/search/heavy?q=electronics	284	0	16000	29000	35000	16885.76	1523	42945	2161.51	0.6	0
GET	/products/search/heavy?q=home	304	0	17000	30000	44000	18080.95	3545	49906	1921.63	2.1	0
GET	/products/search/heavy?q=product	291	0	17000	29000	42000	17900.84	552	53971	1978.62	1.4	0
GET	/products/search/light?q=alpha	794	0	3100	9200	15000	3788.59	60	16582	1094.96	3.4	0
GET	/products/search/light?q=beta	796	0	3100	8600	12000	3666.73	40	18466	1044.98	3.3	0
GET	/products/search/light?q=books	791	0	3000	8900	12000	3680.95	34	16252	1044.98	3.3	0
GET	/products/search/light?q=electronics	818	0	3000	8700	11000	3566.17	38	16393	1094.99	3.7	0
GET	/products/search/light?q=home	772	0	2900	8800	13000	3510.37	60	14576	975	2.6	0
GET	/products/search/light?q=product	792	0	3000	8900	12000	3556.65	76	15640	1971.99	2.8	0
	Aggregated	6492	0	4100	23000	32000	7303.96	34	54413	1431.22	26.8	0







B. Inventory dependency problem

In the failure version, `/products/search/inventory` synchronously called a flaky third-party inventory function, where **70%** of calls slept for **3 seconds** and then

returned an error.

This produced a very unstable endpoint:

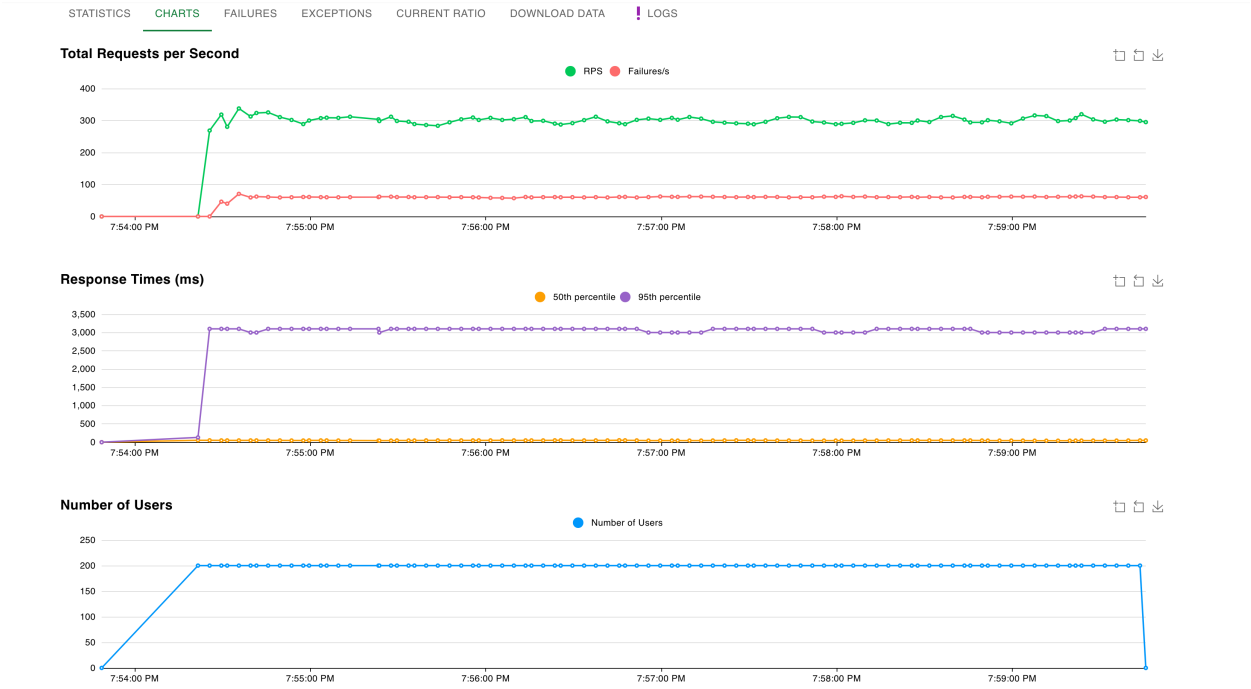
- `median`: **3000 ms**
- `p95`: **3100 ms**
- `average`: about **2175–2181 ms**

It also produced a high error rate. For example:

- `alpha`: **3259 / 4646** failed
- `beta`: **3344 / 4765** failed
- `books`: **3311 / 4717** failed

So the failure rate was roughly **70%**, which matches the simulated dependency behavior. At the aggregate level, the workload reached about **295.1 RPS** with around **60.8 failures/s**.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search/inventory?q=alpha	4646	3259	3000	3100	3100	2177.28	79	3456	401.94	14.3	9.3
GET	/products/search/inventory?q=beta	4765	3344	3000	3100	3100	2177.14	81	3383	386.72	14	10.3
GET	/products/search/inventory?q=books	4717	3311	3000	3100	3100	2176.95	81	3433	386.59	15.9	11.5
GET	/products/search/inventory?q=electronics	4567	3209	3000	3100	3100	2180.95	81	3457	400.77	13.3	9.6
GET	/products/search/inventory?q=home	4670	3274	3000	3100	3100	2175.43	82	3461	366.46	16.5	11
GET	/products/search/inventory?q=product	4623	3213	3000	3100	3100	2157.35	80	3448	675.81	13.4	9.1
GET	/products/search/light?q=alpha	11538	0	43	120	140	52.27	29	465	1094.97	35.5	0
GET	/products/search/light?q=beta	11594	0	43	120	140	52.85	25	461	1044.97	34.3	0
GET	/products/search/light?q=books	11851	0	43	120	140	52.76	28	466	1044.96	37.3	0
GET	/products/search/light?q=electronics	11807	0	43	120	140	52.79	26	456	1094.97	35	0
GET	/products/search/light?q=home	11594	0	43	120	140	53.1	27	454	974.98	31.9	0
GET	/products/search/light?q=product	11569	0	43	120	140	53.64	27	459	1971.95	33.7	0
	Aggregated	97941	19610	47	3100	3100	659.09	25	3461	984.04	295.1	60.8



2. Fixes Applied

A. Bulkhead for `/heavy`

To isolate expensive requests, I added a `semaphore`-style concurrency guard:

- a buffered channel with capacity **2**
- only **2** heavy requests are allowed to run at the same time
- if the heavy lane is full, the service immediately returns `429 Too Many Requests`

This is a `bulkhead` design because it prevents expensive work from consuming unlimited service capacity. It is also a `fail fast` behavior because overload is rejected immediately instead of being allowed to pile up.

B. Circuit breaker for `/inventory`

To protect the service from the unstable dependency, I added a simple `circuit breaker`:

- after **3 consecutive failures**, the breaker opens
- once open, it stays open for **10 seconds**

- while open, the service skips the inventory call and returns a degraded search response instead of failing the whole request

This prevents the service from repeatedly calling an unhealthy dependency and greatly improves availability.

3. Improved Results

A. Heavy workload after `bulkhead`

After adding the `bulkhead`, the behavior changed dramatically.

The `/products/search/light` endpoints improved from multi-second latency to:

- `median`: about **110 ms**
- `p95`: about **290 ms**
- `average`: about **136–137 ms**

The aggregated throughput increased from **26.8 RPS** to about **1544.1 RPS**.

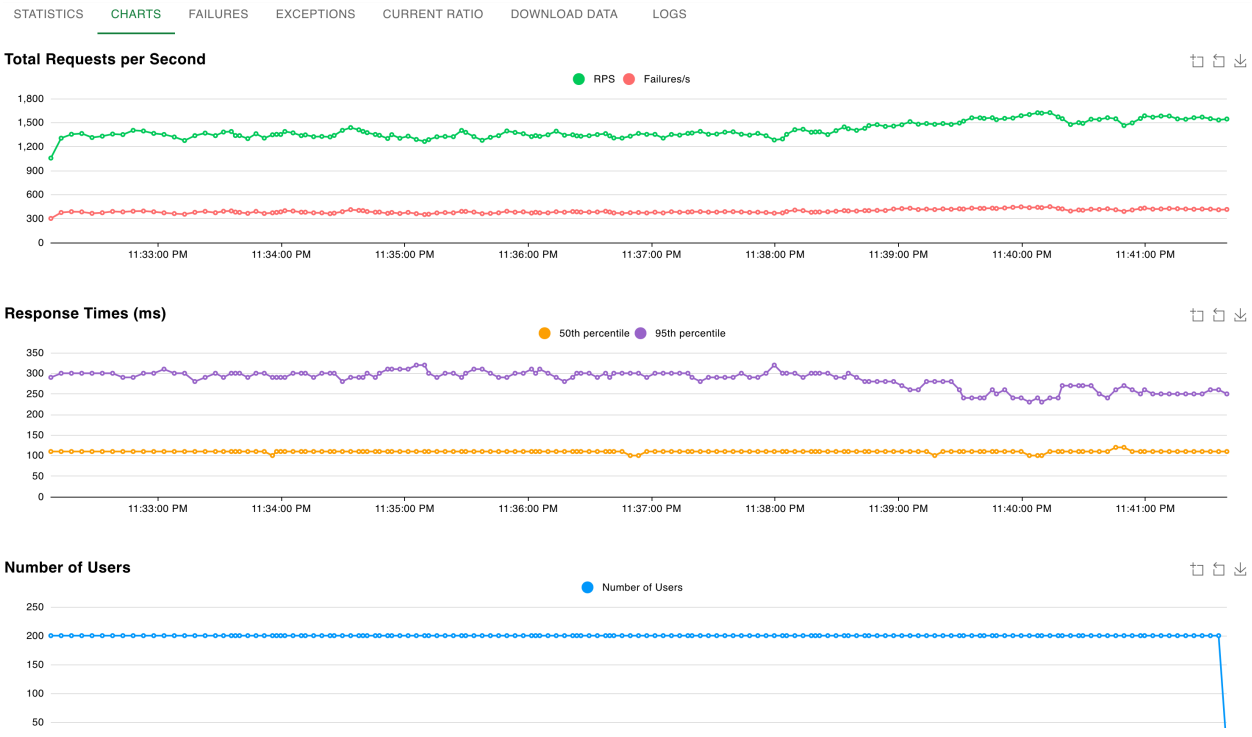
The `/products/search/heavy` endpoints now showed very high failure counts, but these failures were expected and intentional. For example:

- `alpha`: **37297 / 38223** failed
- `beta`: **37497 / 38401** failed
- `home`: **37870 / 38804** failed

These are not random crashes; they are deliberate fast `429` rejections used to protect the system. In exchange, the requests that were rejected failed quickly, with heavy endpoint latency around:

- `median`: about **110 ms**
- `p95`: about **300 ms**

This demonstrates the main trade-off of the fix: **heavy-request availability was reduced, but overall system health and light-request performance improved massively**.



B. Inventory workload after `circuit breaker`

After adding the `circuit breaker`, the inventory path became much more stable.

The `/products/search/inventory` endpoints improved to:

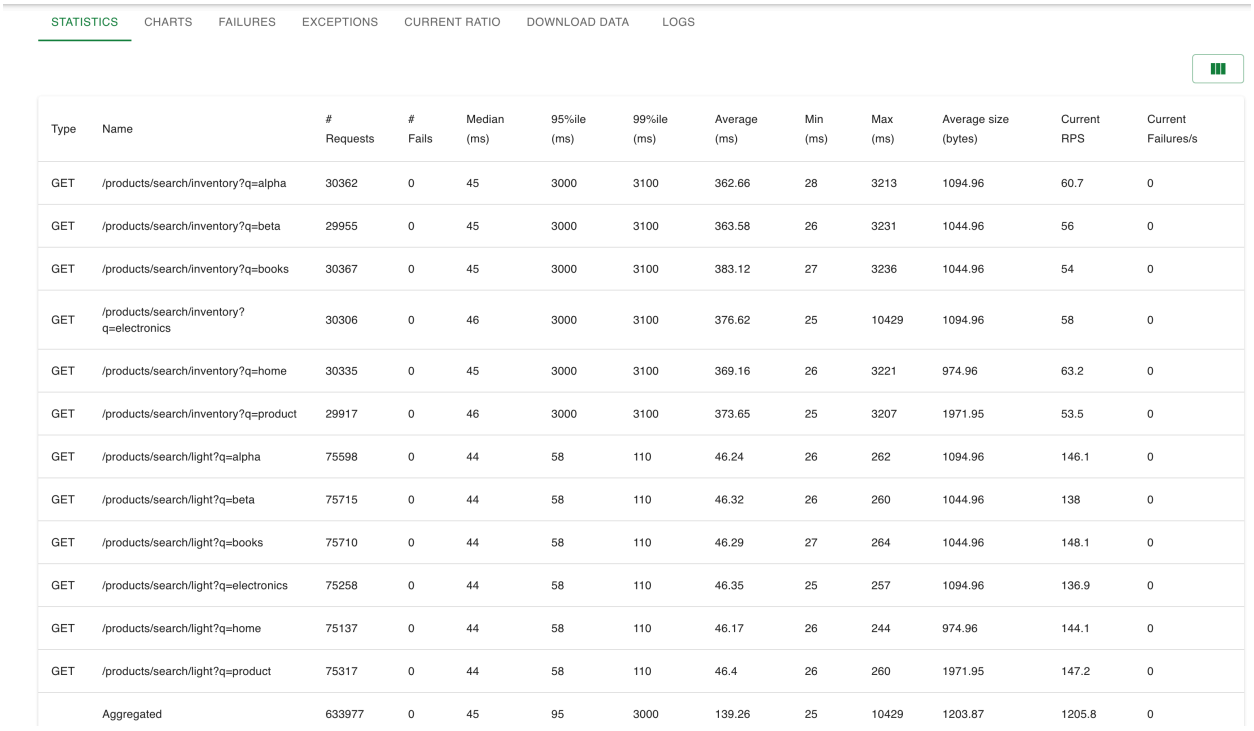
- `median`: about **45–46 ms**
- `average`: about **363–383 ms**
- `fails`: **0** for all shown inventory requests

At the same time, the `/products/search/light` endpoints remained very stable:

- `median`: about **44 ms**
- `p95`: about **58 ms**
- `average`: about **46 ms**

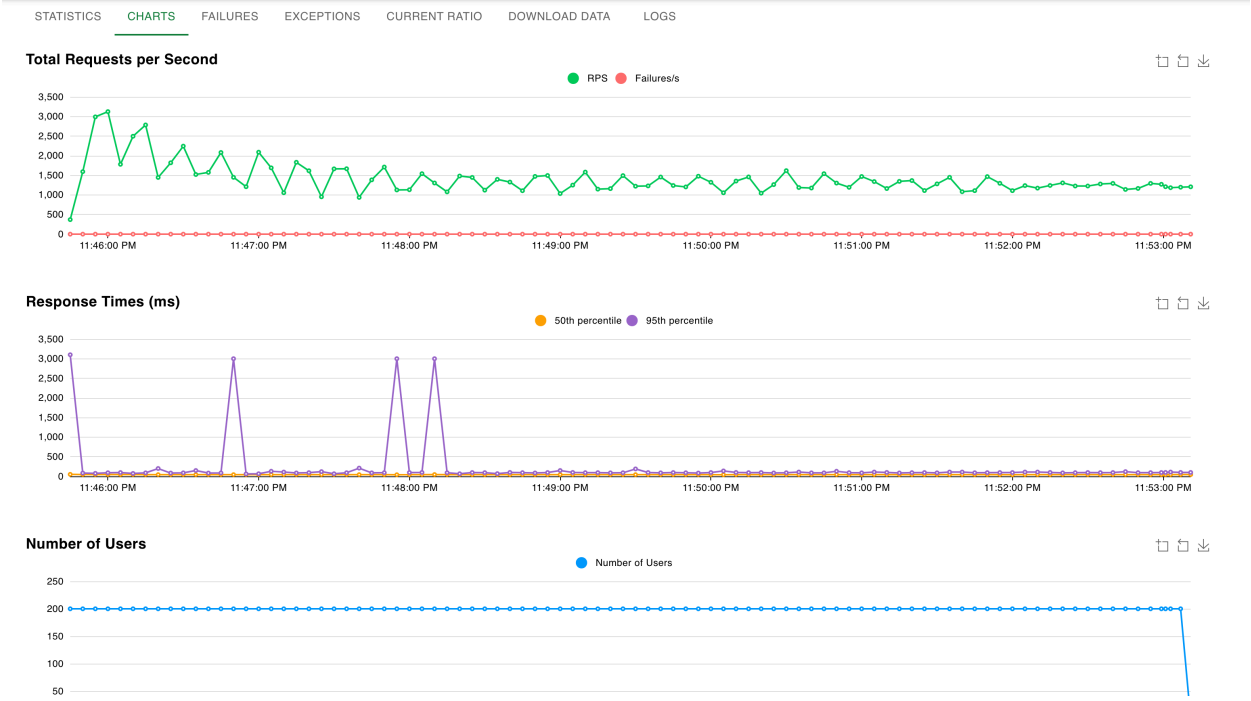
The aggregated throughput increased from **295.1 RPS** to about **1205.8 RPS**, and `Current Failures/s` dropped to **0**.

One important detail is that inventory `p95` and `p99` still reached about **3000 ms** and **3100 ms**. This indicates that some requests still hit the slow dependency before the breaker opened, so the `circuit breaker` greatly improved average behavior and availability, but it did not fully eliminate tail latency.



STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/search/inventory?q=alpha	30362	0	45	3000	3100	362.66	28	3213	1094.96	60.7	0
GET	/products/search/inventory?q=beta	29955	0	45	3000	3100	363.58	26	3231	1044.96	56	0
GET	/products/search/inventory?q=books	30367	0	45	3000	3100	383.12	27	3236	1044.96	54	0
GET	/products/search/inventory?q=electronics	30306	0	46	3000	3100	376.62	25	10429	1094.96	58	0
GET	/products/search/inventory?q=home	30335	0	45	3000	3100	369.16	26	3221	974.96	63.2	0
GET	/products/search/inventory?q=product	29917	0	46	3000	3100	373.65	25	3207	1971.95	53.5	0
GET	/products/search/light?q=alpha	75598	0	44	58	110	46.24	26	262	1094.96	146.1	0
GET	/products/search/light?q=beta	75715	0	44	58	110	46.32	26	260	1044.96	138	0
GET	/products/search/light?q=books	75710	0	44	58	110	46.29	27	264	1044.96	148.1	0
GET	/products/search/light?q=electronics	75258	0	44	58	110	46.35	25	257	1094.96	136.9	0
GET	/products/search/light?q=home	75137	0	44	58	110	46.17	26	244	974.96	144.1	0
GET	/products/search/light?q=product	75317	0	44	58	110	46.4	26	260	1971.95	147.2	0
	Aggregated	633977	0	45	95	3000	139.26	25	10429	1203.87	1205.8	0



Conclusion

This experiment showed two different resilience improvements:

- For expensive search traffic, a `bulkhead` successfully isolated heavy work and protected the rest of the system. The result was a huge improvement in `light` latency and total `RPS`, at the cost of intentionally rejecting excess heavy traffic with fast `429` responses.
- For the unstable inventory dependency, a `circuit breaker` eliminated the large visible failure rate and significantly improved throughput and median latency by preventing repeated calls to an unhealthy dependency.

Overall, the improved system behaved much more predictably under load. Instead of failing slowly or allowing expensive requests to dominate the service, it either **isolated overload** or **degraded gracefully**.

Future Improvement

A useful next step would be to add a true request `timeout` using `context.Context` around the inventory call. That would likely reduce the remaining `p95/p99` spikes even further.